other failure cases that might cause the workflow to return failure, I would not write an end-to-end test for each one because the controller behavior remains the same.

This test has a workflow with empty strings and checks an as-yet-nonexistent success? method to determine if the workflow has failed:

test_first/06/spec/workflows/creates_project_spec.rb

```
describe "failure cases" do
  it "fails when trying to save a project with no name" do
    creator = CreatesProject.new(name: "", task_string: "")
    creator.create
    expect(creator).not_to be_a_success
  end
end
```

(At this point, you could also refactor the tests to change the original let block for creator to take the name as something defined by later let calls, the same way you already do for task_string, so as to keep this test in line with the pattern of other tests.)

You now have two failing tests. And if you'll indulge a quick side note, there's an easy way to get RSpec to focus on just running the failures, which can be very helpful in a situation like this.

Uncomment the following line in the spec/spec_helper.rb:

```
config.example_status_persistence_file_path = "spec/examples.txt"
```

This tells RSpec to persist runtime information at spec/examples.txt. You'll also want to add that file to your .gitignore listing.

Now, if you rerun RSpec it generates the file spec/examples.txt. Here are the first few lines of that file:

```
| example_id                                  | status | run_time        |
|:--------------------------------------------|:-------|:----------------|
| ./spec/features/add_project_spec.rb[1:1] | passed | 0.17632 seconds |
| ./spec/features/add_project_spec.rb[1:2] | failed | 0.06486 seconds |
| ./spec/models/project_spec.rb[1:1:1]     | passed | 0.00191 seconds |
```

It's a list of every spec in the system, whether it most recently passed or failed, and the amount of time it took to run the last time it ran. The numbers for each spec are the order of the spec in its example group. You'll notice here that the first line, which is the first test in the feature spec that successfully creates a project, is much, much slower than the other tests.

Anyway, now that you have this information, you have two useful command-line options for RSpec. You can run your tests as rspec --only-failures, and RSpec